

python - Day - 8 (Beginner)

Until now, you used function like `print()`, `len()` etc

Today is where python levels up - you learn how to create flexible functions using parameters.

It's where python shifts from script writing

↓
Software thinking

(1) Why functions Exist (Deep Concept)

Before functions:

```
print("Hello")  
print("Hello")  
print("Hello")
```

Problem:

- Repetition
- Hard to Update
- Error-prone

with functions:

```
def say_hello():  
    print("Hello")
```

One definition → infinite reuse

Software Engineering principle

- Abstraction → hide complexity
- Encapsulation → group logic
- Maintainability → change once, update everywhere

(2) Function Anatomy (Internal Mechanics)

```
def greet (name):
    print(f"Hello {name}")
```

what python actually does:

- (1) Stores greet in memory
- (2) Associates it with a block of code
- (3) waits until it is called
- (4) On call → assigns name = argument
- (5) Executes body
- (6) Returns control
- (*) function code does NOT run until called

(3) Parameters vs Arguments (Exam Confusion Zone)

Parameter

Argument

Meaning
Variable inside funⁿ definition
Actual value passed during
function call

```
def add(a, b):    # a, b → parameters
    return a + b
```

```
add(3, 5)        # 3, 5 → arguments
```

- in C/C++ you think "formal vs actual parameters"
- python uses the same concept but more flexibly

(4) Multiple parameters - Stack Mapping Model

```
def func(x, y)
    print(x, y)
```

func(10, 20)

Memory mapping

x → 10

y → 20

Wrong Order

func(20, 10)

→ logical bug, not syntax error

(5) Keyword Arguments (python Superpower)

```
func(y=20, x=10)
```

why python developers love this

- Self-documenting code
- fewer bugs
- Order independent

(6) Default parameters (Important for professional code)

```
def greet(name = "User"):  
    print(f"Hello {name}")
```

Calling

```
greet() # Hello user
```

```
greet("Mohit") # Hello Mohit
```

Rules:

- Default parameters must come after non-default

```
def f(a, b=5): # valid
```

```
def f(a=5, b): # invalid
```

(7) Return vs print (Hidden trap)

print

```
def add(a, b):
```

```
    print(a+b)
```

- Displays Output
- Returns None

Return

```
def add(a, b):
```

```
    return a+b
```

- Sends value back
- Can be stored, reused, tested

(8) Scope

```
def test():
```

```
    x=10
```

```
    print(x) # (x) Error
```

Scope rules:

- variables inside function → local
- variables Outside → global
- functions create a new Scope

(*)(* Mini project - Caesar Cipher (*)(*)

This project teaches algorithmic thinking, not encryption

Core Steps:

- (1) Take input text
- (2) Loop through each character
- (3) Find its index
- (4) Shift index
- (5) Wrap around if needed
- (6) Rebuild string

Final Day 8 (Keytakeaways)

- Functions = reusable logic blocks
- Parameters $\neq$ arguments
- Keyword arguments reduce bugs
- % Enable cycle logic
- Strings are immutable
- Python favors readability over cleverness
- Real programs handle edge cases.